

Natural language processing

Ask PDF

Finn Årup Nielsen

DTU Compute, Technical University of Denmark

March 25, 2026

Ask PDF

Question-answering task for a long text in a binary format.

How to convert the file (exercise 2 “PDF-to-sentences”)

How to handle long contexts that does not fit in the prompt of a normal LLM (exercises “Retrieval-augmented generation” and “text to SPARQL”)

Methods

Standard RAG

Agentic RAG with ReAct

Recursive Language Model (RLM)

Standard RAG

Convert the PDF to a text format, e.g., markdown.

Chunk the markdown in overlapping windows sufficiently short to fit in the prompt of the LLM.

Add the chunks to a dense and/or sparse information retrieval system.

For the query search for chunks and combine them with prompt, question to generate the answer.

Almost like the RAG exercise except for the PDF and the chunking.

ReAct with TEI tools

ReAct where the tools are related with operations on the TEI XML-converted document.

```
1 class TEIDocument:
2     def __init__(self, xml_string):
3         self.root = ET.fromstring(xml_string)
4         self.ns = {"tei": "http://www.tei-c.org/ns/1.0"}
5
6     def get_title(self):
7         title = self.root.find("./tei:title", self.ns)
8         return "".join(title.itertext()).strip() if title is not None else ""
9
10    def get_section_titles(self) -> list[str]:
11        return [
12            "".join(h.itertext()).strip()
13            for h in self.root.findall("./tei:head", self.ns)
14        ]
```

These functions may include common sparse/dense information retrieval.

Combines element of RAG and text-to-query exercises.

ReAct with TEI tools

Set up of DSPy ReAct agent with tool for TEI XML

```
1 def make_tools(doc: TEIDocument):
2     return [
3         doc.get_title,
4         doc.get_section_titles,
5         doc.get_sentences,
6         doc.get_sentences_in_section,
7     ]
8 ...
9 doc = TEIDocument(xml_string)
10 tools = make_tools(doc)
11 ...
12 agent = dspy.ReAct(
13     AskTEI,
14     tools=tools,
15     max_iters=5,
16 )
```

Recursive Language model

Recursive Language Models (RLM) makes the context available in a sandboxed execution environment and the LLM is able to generate code that is executed in the sandbox.

RLM is available in DSPy as `dspy.RLM` using a `dspy.PythonIntepreter` which relies on a somewhat fragile combination of Deno (JavaScript runtime) and Pyodide (Python running in Javascript).

Only the Python Standard Library is available in Pyodide, — not, e.g., Numpy.

Sandbox with RLM

Question “What is the title”

```
1 > What is the title
2 2026/03/24 23:55:06 INFO dspy.predict.rlm: RLM iteration 1/20
3 Reasoning: The question asks for the title of the document. The 'context' variable
  contains the TEI XML content. I need to parse the XML to find the '<title>' tag
  and extract its text. I will use regular expressions to achieve this since I don'
  t have an XML parser available. I'll look for the '<title>' tag and extract the
  content within it.
4 Code:
5 '''python
6 import re
7
8 title_match = re.search(r'<title[^>]*>(.*?)</title>', context)
9 if title_match:
10     title = title_match.group(1)
11     print(title)
12 else:
13     print("Title not found.")
14 '''
```

A small Python program is created dynamically by the LLM (CampusAI).

Sandbox with RLM

The LLM receives the result of the Python execution.

```
1 2026/03/24 23:55:09 INFO dspy.predict.rlm: RLM iteration 2/20
2 Reasoning: The previous code successfully extracted the title "Natural language
   processing" from the context using regular expressions. Now, I need to submit
   this as the answer to the question.
3 Code:
4 '''python
5 SUBMIT("Natural language processing")
6 '''
7 Natural language processing
```